데이터 분석을 위한

Pandas

류영표 강사
ryp1662@gmail.com



Youngpyo Ryu. All rights reserved.
Unauthorized reproduction or distribution is prohibited.

류 영 표

AI & Applied Mathematics Researcher · Lecturer

ryp1662@gmail.com



학력

- 동국대학교 일반대학원 응용수학 석사 수료 (2016-2019)
- 동국대학교 수학과 졸업 (2010-2016)

심사 및 전문가 자문 활동

- 인공지능 학습용 데이터 구축 사업 품질검증 전문가 자문위원 (2023)
- 2023 신뢰할 수 있는 인공지능 개발 안내서(안)_고도화 전문위원
- 2023 자율주행/의료/공공사회 분야 신뢰할 수 있는 인공지능 개발 안내서(안) 자문위원

연구 및 프로젝트

- 부산대학교 병원: 개인 맞춤형 당뇨병 예방·관리 인공지능 시스템 외주 프로젝트 (2022)
- 연세대학교 병원: 영상의학과 FLASK 고도화 연구 (2023, 2024)
- 산업수학 스터디 그룹 (국가수리과학연구소): 피부암/유방암 분류, 유전자 정보 분석
- 폐플라스틱 이미지 객체 검출 경진대회 3위 (2021)
- 동국대학교 수학과 연구원(2025.10 ~ / 양자 AI, XAI 연구 진행 중)

논문

- Quantum computing for the optimization of CT image reconstruction, IEEE ICTC 2022

㈜ 헬로월드랩스 전담교육팀 과장

강의 및 교육 활동

대학 강의

- 고려대학교, 인천대학교, 동양미래대학교, 목포대학교
- 이젠 아카데미: 파이썬/ADSP 과정

기업 교육

- 현대자동차 연구원
- 한화, 하나금융, 한전 KDN, IBK 기업은행, 아가방앤컴퍼니, IM증권

공공/국비 과정

- [Est Camp] AI 퀀트 캠프 1기
- 보건·바이오 AI 활용 전문가 양성과정 (1-3기)
- 국제인공지능올림피아드 한국 대표 멘토
- 충청 ICT 취창업 역량강화 프로그램 (SQLD)

아카데미 & 부트캠프

- 모두의연구소 Aiffel 퍼실리테이터 (1기)
- Upstage AI Boostcamp 멘토 (1-6기)
- 패스트캠퍼스/멀티캠퍼스 조교 및 멘토
- 공개SW개발자대회 멘토

Pandas | 데이터 분석을 위한 라이브러리

Pandas란?

pandas

- 이름은 **PANel Data**의 약자로,
  금융 경제학 용어에서 유래되었음.
- 판다는 데이터 조작 및 분석을 위해 설계되었음.
- 빠르고 강력한 데이터 구조를 제공하여
  대규모 데이터 세트를 **효율적으로 처리**할 수 있습니다.

PANel Data란?

여러 개의 개체(예: 기업, 국가, 개인 등)를 시간에 따라 추적한 데이터를 의미합니다.

예시

여러 기업의 재무 정보(기업 × 시간)
여러 국가의 경제 지표(국가 × 시간) 등



데이터 처리, 분석용 라이브러리

금융적인 데이터 처리와 분석을 위한 파이썬 라이브러리

다양한 형태의 데이터 처리

표 형식 데이터, 시계열 데이터 등 다양한 형태의 데이터를 다루는 데 초점
(csv, text files, excel, SQL database, ···)

numpy 기반 계산 스타일

numpy의 배열 기반 계산 스타일을 많이 사용

Pandas의 주요 특징

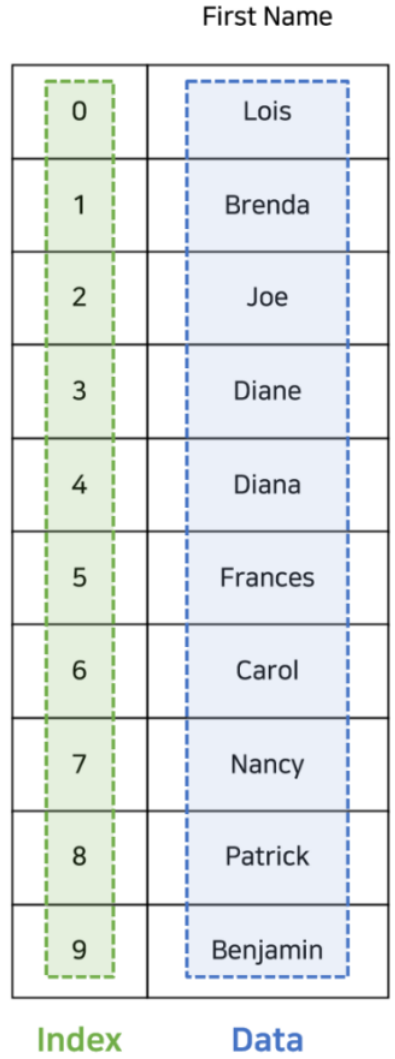
1	다양한 데이터 타입 처리	정수, 실수, 문자열, Python 객체 등 다양한 타입의 데이터를 처리할 수 있음.	예) 정수, 실수, 문자열, Python 객체 등을 처리할 수 있음.
2	누락된 데이터 처리	누락된 데이터를 쉽게 처리할 수 있음.	예) 누락된 데이터를 특정 값으로 채우거나 누락된 데이터를 표함하는 행이나 열을 삭제할 수 있음.
3	데이터 병합	여러 데이터프레임을 하나로 병합하거나 조인할 수 있는 기능을 제공함.	예) 여러 테이블(데이터프레임)을 다양한 방식으로 병합/조인
4	피벗 테이블 생성	피벗 데이블을 쉽게 생성할 수 있음. 피벗 테이블은 데이터를 요약하고 분석하는 데 유용함.	예) 데이터를 그룹화하여 요약/집계 결과를 표 형태로 변환
5	시계열 데이터 처리	시계열 데이터를 다루는 데 매우 유용한 도구들을 제공함.	예) 날짜 인덱싱, 리샘플링, 이동 평균, 시차 데이터 생성 등

Series와 DataFrame

series

	Date	kospi	kosdaq	gold_fut_132030	Bond_273130
0	2020. 1. 2 오후 3:30:00	2175.17	674.02	10845	108215
1	2020. 1. 3 오후 3:30:00	2176.46	669.93	11000	108565
2	2020. 1. 6 오후 3:30:00	2155.07	655.31	11245	108745
3	2020. 1. 7 오후 3:30:00	2175.54	663.44	11180	108400
4	2020. 1. 8 오후 3:30:00	2151.31	640.94	11360	108270
5	2020. 1. 9 오후 3:30:00	2186.45	666.09	11055	107980
6	2020. 1. 10 오후 3:30:00	2206.39	673.03	11035	107760
7	2020. 1. 13 오후 3:30:00	2229.26	679.22	11080	107695
8	2020. 1. 14 오후 3:30:00	2238.88	678.71	10975	107860

DataFrame



Series

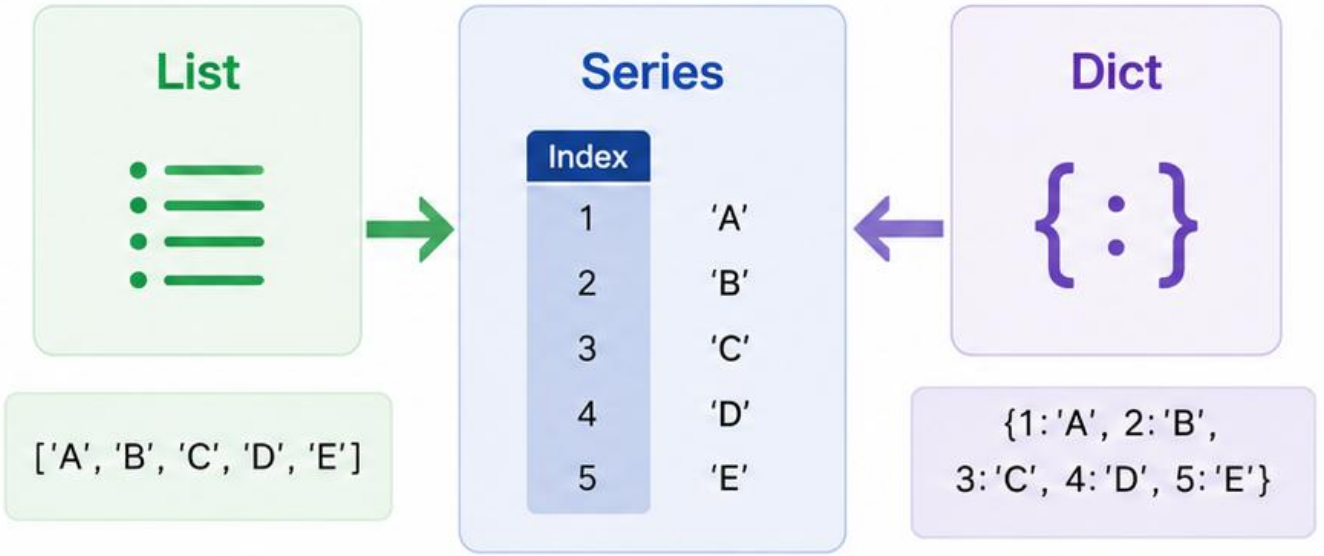
pandas

Series는 **1차원 배열**과 유사한 자료구조로,
각 데이터(값)에 **인덱스(라벨)**가 부여된 형태입니다.

Series 구조

Index (인덱스)	Data (값)
1	'A'
2	'B'
3	'C'
4	'D'
5	'E'

Series는 다양한 파이썬 자료형으로부터 생성할 수 있습니다.



핵심 포인트

Series는 **인덱스**로 데이터에 의미를 부여하여, 데이터 선택, 정렬, 연산 등을 더 쉽고 직관적으로 수행할 수 있게 해줍니다.

Series

pandas

Series는 **1차원 배열(백터)** 같은 자료구조로,
각 값(value)에 **인덱스(index)**가 대응되는 구조입니다.

1 Series 생성하기

- list, numpy, dictionary 동을 이용하여 Series 생성 가능

```
pd.Series(data, index, dtype, name)
```

index : 각 row의 이름, 기본적으로는 숫자로 되어있음

dtype : numpy의 dtype과 동일

name : 각 column의 이름, 기본값이 없음

예시

```
import pandas as pd
s = pd.Series([10, 20, 30],
              index=['a', 'b', 'c'],
               dtype='int64',
               name='score')
s
```

```
In [ ]:  a    10
         b    20
         c    30
         Name: score, dtype: int64
```

인덱스(index)
각 row의 이름

값(value)
데이터

이름(name)과 타입(dtype)

핵심 포인트
Series는 **1차원 데이터**와 **인덱스**로 구성되어 있어, 데이터를 보다 쉽게 선택, 정렬, 연산할 수 있도록 도와줍니다.

DataFrame | Pandas의 2차원 데이터 구조

pandas

DataFrame이란?

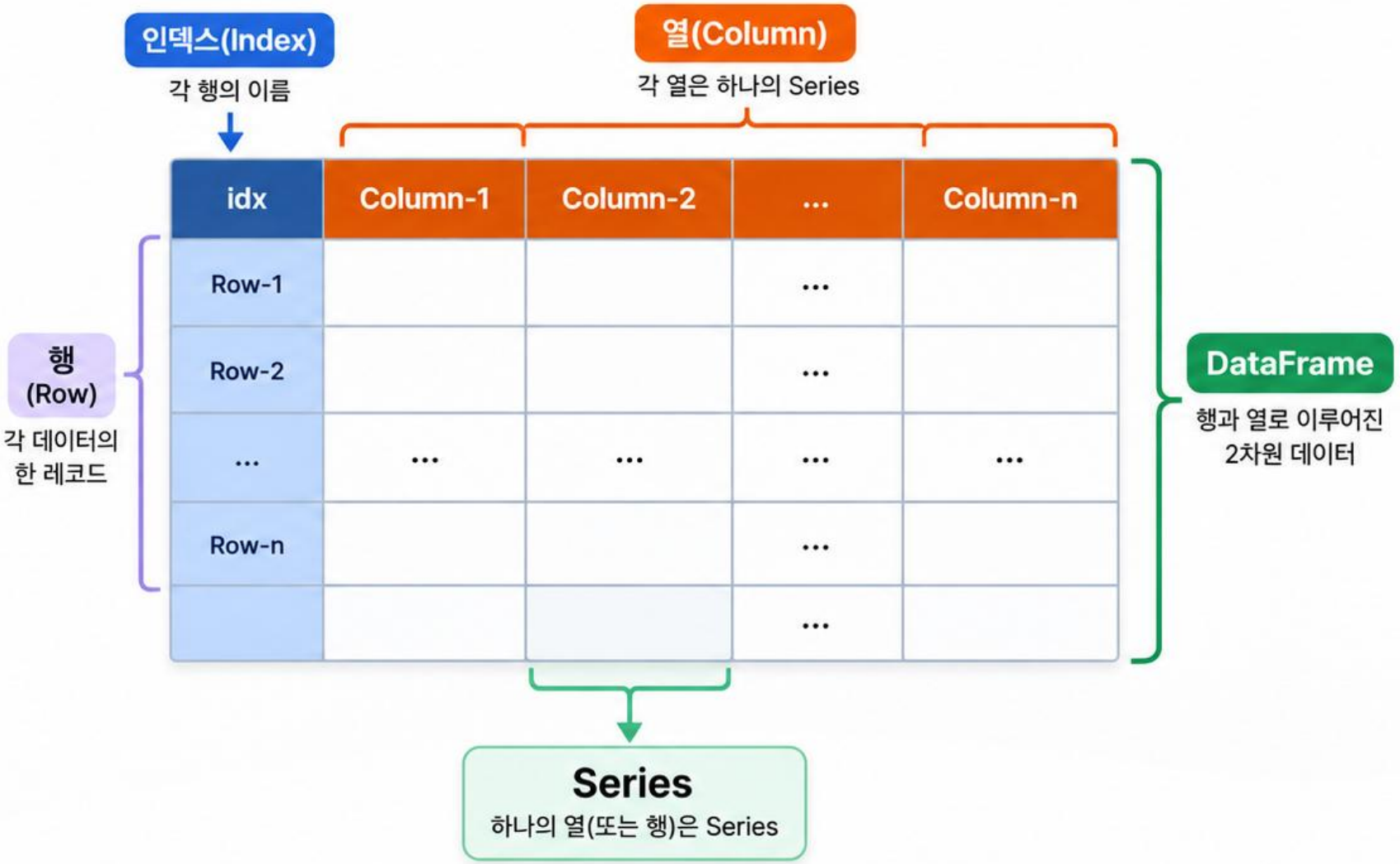
행(Row)과 **열(Column)**로 구성된
2차원 데이터 구조입니다.

- 각 열은 Series 형태
- 서로 다른 타입의 데이터 저장 가능
- 라벨(인덱스) 기반으로 데이터 정렬 및 선택
- 엑셀과 유사한 표 형태의 구조

핵심 포인트

DataFrame은 **여러 개의 Series**가
나란히 모여 있는 표 형태의 데이터 구조입니다.

Pandas DataFrame 구조



DataFrame | 2차원 표 형태의 데이터 구조

DataFrame은 행(row)과 열(column)로 이루어진 2차원 표 형태의 데이터 구조입니다.

1 DataFrame의 특징

- 표(스프레드시트)와 같은 데이터 구조
- index가 같은 Series가 여러 개 모여 있는 형태

2 DataFrame 생성하기

- dictionary나 numpy 배열을 이용하여 생성

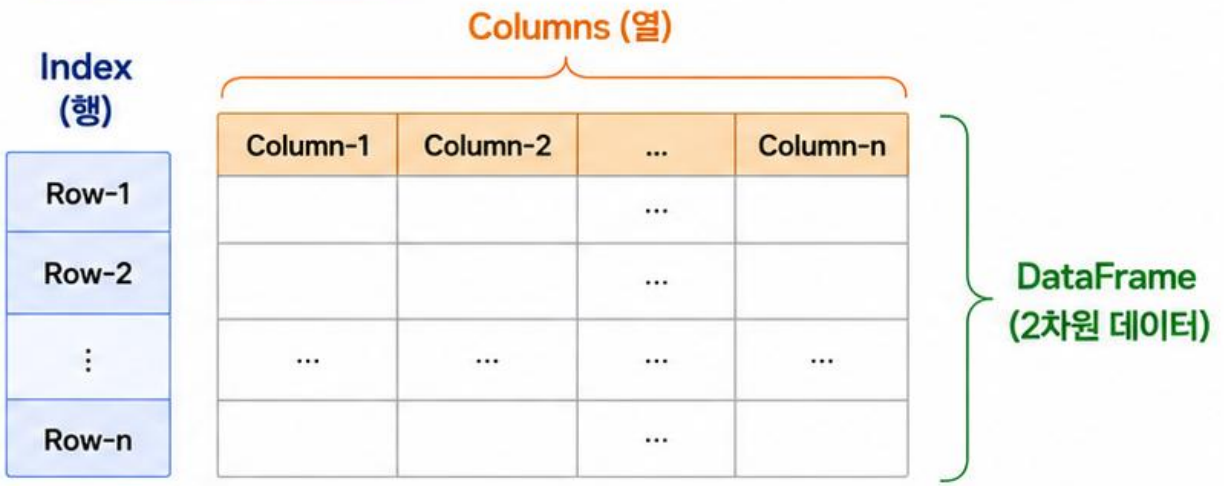
pd.DataFrame(data, index, dtype, columns)

index : 각 row의 이름, 기본적으로는 숫자로 되어있음

dtype : numpy의 dtype과 동일

columns : 각 column의 이름, 기본값이 없음

DataFrame 구조



예제

```
import pandas as pd
data = {
   'name': ['Kim', 'Lee', 'Park'],
   'age': [23, 21, 24],
   'score': [90, 85, 95]
}
df = pd.DataFrame(data,
         index=['S1', 'S2', 'S3'],
         columns=['name', 'age', 'score'])
df
```

	name	age	score
S1	Kim	23	90
S2	Lee	21	85
S3	Park	24	95

데이터 다루기

1 데이터 로딩

pandas file 파싱 함수	설명
read_csv	파일, url, 파일과 유사한 걱체로부터 구분된 데이터를 읽어옴. 데이터 구분자는 쉼표(,)가 기분
read_excel	엑셀(xls, xlsx)에서 표 형식의 데이터를 읽어옴.
read_json	JSON 문자열에서 데이터를 읽어옴.
read_pickle	파이썬 피클 포맷으로 저장된 객체를 읽어옴.

2 데이터 저장

pandas file 저장 함수	설명
to_csv	csv 형식으로 저장
to_json	JSON 형식으로 저장
to_pickle	파이썬 피클 포맷으로 저장

Series

1



integer-location based

- numpy indexing과 유사하게 동작함

2



label-location based

- label(index)를 기준으로 동작함

3



Series.iloc

- integer-location based property

4



Series.loc

- label-location based property

Series 생성

```
import pandas as pd
import numpy as np
```

```
obj = pd.Series([0,1,2,3,4,5,6,7], index=['a','b','c','d','e','f','g','h'], dtype = 'int64')
obj
```

```
a    0
b    1
c    2
d    3
e    4
f    5
g    6
h    7
dtype: int64
```

```
# index
obj2 = pd.Series(['a', 'b', 'c'], index = ['i1','i2','i3'])
obj2
```

```
i1    a
i2    b
i3    c
dtype: object
```

Series

1. integer-location based

- numpy indexing과 유사하게 동작함

```
[10] obj
```

```
a    0
b    1
c    2
d    3
e    4
f    5
g    6
h    7
dtype: int64
```

```
[5]  obj[3]
```

→ numpy의 indexing과 유사하게 동작

```
3
```

```
[6]  obj[-1]
```

→ 리스트에 위치(int)를 넘겨주면 그위치에 해당되는 자료들을 index(label)값과 함께 반환

```
7
```

```
[7]  obj[[1,3,5]]
```

```
b    1
d    3
f    5
dtype: int64
```

Series

1. integer-location based

- numpy indexing과 유사하게 동작함

```
[10] obj
```

```
a    0
b    1
c    2
d    3
e    4
f    5
g    6
h    7
dtype: int64
```

```
obj[1:3]
```

→ slicing 또한 numpy slicing과 유사하게 동작

```
b    1
c    2
dtype: int64
```

```
[9]  # boolean
     obj[obj<3]
```

→ boolean indexing 처럼 작동

```
a    0
b    1
c    2
dtype: int64
```

2. label-location based

- label(index)를 기준으로 동작함

```
[10] obj
```

```
a    0
b    1
c    2
d    3
e    4
f    5
g    6
h    7
dtype: int64
```

```
[13] obj['c']
```

```
2
```

→ indexing 처럼

```
[14] obj.c
```

```
2
```

→ method나 attribute처럼

```
[15] obj[['e','c']]
```

```
e    4
c    2
dtype: int64
```

→ 리스트에 담아서 여러개의 자료를 indexing

Series

2. label-location based

- label(index)를 기준으로 동작함

```
[10] obj
```

```
a    0
b    1
c    2
d    3
e    4
f    5
g    6
h    7
dtype: int64
```

```
[16] obj['a':'c']
```

```
a    0
b    1
c    2
dtype: int64
```

주의
라벨값을 이용해 슬라이싱을
하면 끝점까지 모두 포함

Series

2. label-location based

- label(index)를 기준으로 동작함

```
[10] obj
```

```
a    0
b    1
c    2
d    3
e    4
f    5
g    6
h    7
dtype: int64
```

```
[17] obj['d':'e']=100
```

→ slicing을 이용해 값을 할당하면 값이 잘 변경되는 것을 확인할 수 있음.

```
obj
```

```
a      0
b      1
c      2
d    100
e    100
f      5
g      6
h      7
dtype: int64
```

Series

3. Series.iloc

- integer-location based property

```
[10] obj

a    0
b    1
c    2
d    3
e    4
f    5
g    6
h    7
dtype: int64
```

```
[27] obj.iloc[2]

2
```

→ 위치에 해당하는 값만

```
[28] obj.iloc[[2]]

c    2
dtype: int64
```

→ 리스트에 담아 위치를 넘기면 index(label)과 함께 반환

```
[29] obj.iloc[1:4]

b    1
c    2
d    3
dtype: int64
```

→ 위치기반 슬라이싱

Series

4. Series.loc

- label-location based property

```
[10] obj
```

```
a    0
b    1
c    2
d    3
e    4
f    5
g    6
h    7
dtype: int64
```

```
obj.loc['a':'c']
```

```
a    0
b    1
c    2
dtype: int64
```

→ 라벨로 슬라이싱을 하기 때문에 끝점까지 포함

iloc와 loc 비교

Pandas에서 데이터를 선택할 때 사용하는 대표적인 인덱싱 방식입니다.

구분	기준	예시	끝점 포함
iloc	정수 위치 (0부터 시작)	`s.iloc[1:3]`	**미포함** (마지막 위치는 포함하지 않음)
loc	인덱스 라벨 (라벨 이름 기준)	`s.loc['b'::'d']`	**포함** (마지막 라벨까지 포함)

예시 Series

```
s = pd.Series(
    [10, 20, 30, 40, 50],
    index=['a', 'b', 'c', 'd', 'e']
)
```

인덱스(라벨)	a	b	c	d	e
값	10	20	30	40	50

iloc (정수 위치 기준)

```
s.iloc[1:3]    # 1번째부터 3번째 전까지
```

	a	b	c	d	e
	10	20	30	40	50
	0	1	2	3	4

결과 →

b	c
20	30

↳ 마지막 위치(3)는 포함하지 않음

loc (인덱스 라벨 기준)

```
s.loc['b'::'d']    # 'b'부터 'd'까지
```

a	b	c	d	e
10	20	30	40	50

결과 →

b	c	d
20	30	40

↳ 마지막 라벨('d')까지 포함

TIP

- iloc는 파이썬 슬라이싱 규칙(끝점 미포함)을 따릅니다.
- loc는 라벨을 기준으로 하며, 슬라이싱 시 끝 라벨까지 포함합니다.

DataFrame

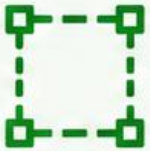
1



indexing

- 컬럼을 기준으로

2



slicing

- 인덱스(라벨)를 기준으로

3



DataFrame.iloc

- integer-location based property

4



DataFrame.loc

- label-location based property

DataFrame

1. indexing

- 컬럼을 기준으로

```
frame = pd.DataFrame(np.arange(24).reshape(4,-1),
                     columns = ['c1', 'c2', 'c3', 'c4', 'c5', 'c6'],
                     index = ['r1', 'r2', 'r3', 'r4'])
frame
```

	c1	c2	c3	c4	c5	c6
r1	0	1	2	3	4	5
r2	6	7	8	9	10	11
r3	12	13	14	15	16	17
r4	18	19	20	21	22	23

```
[7]  frame['c3']
```

```
r1     2
r2     8
r3    14
r4    20
Name: c3, dtype: int64
```

```
[8]  frame.c3
```

```
r1     2
r2     8
r3    14
r4    20
Name: c3, dtype: int64
```

DataFrame

1. indexing

- 컬럼을 기준으로

```
frame
```

	c1	c2	c3	c4	c5	c6
r1	0	1	2	3	4	5
r2	6	7	8	9	10	11
r3	12	13	14	15	16	17
r4	18	19	20	21	22	23

```
frame[['c1', 'c2']]
```

	c1	c2
r1	0	1
r2	6	7
r3	12	13
r4	18	19

DataFrame

2. slicing

- 라벨(row)을 기준으로

```
frame
```

	c1	c2	c3	c4	c5	c6
r1	0	1	2	3	4	5
r2	6	7	8	9	10	11
r3	12	13	14	15	16	17
r4	18	19	20	21	22	23

```
[10] frame['r1':'r2']
```

	c1	c2	c3	c4	c5	c6
r1	0	1	2	3	4	5
r2	6	7	8	9	10	11

```
[11] frame['c1':'c2']
```

	c1	c2	c3	c4	c5	c6

→ 컬럼을 기준으로 슬라이싱을 하게 되면 아무런 값을 가져오지 않음

Q) C1,C2 으로 접근하려면?

```
frame[['c1','c2']]
```

	c1	c2
r1	0	1
r2	6	7
r3	12	13
r4	18	19

DataFrame

3. DataFrame.iloc

- integer-location based property

df.iloc(row, column)

```
frame
```

	c1	c2	c3	c4	c5	c6
r1	0	1	2	3	4	5
r2	6	7	8	9	10	11
r3	12	13	14	15	16	17
r4	18	19	20	21	22	23

```
[23] frame.iloc[[0],[3]]
```

	c4
r1	3

```
[24] frame.iloc[[0,1], 1:4]
```

	c2	c3	c4
r1	1	2	3
r2	7	8	9

DataFrame

4. DataFrame.loc

- label-location based property

df.loc(row, column)

```
frame
```

	c1	c2	c3	c4	c5	c6
r1	0	1	2	3	4	5
r2	6	7	8	9	10	11
r3	12	13	14	15	16	17
r4	18	19	20	21	22	23

```
[25] frame.loc[['r1'],['c4']]
```

	c4
r1	3

```
[26] frame.loc['r1':'r2',['c2','c3','c4']]
```

	c2	c3	c4
r1	1	2	3
r2	7	8	9

→ label-based는 슬라이싱을 하면 끝점까지 포함

산술 연산

메소드	설명
add, radd	덧셈(+)을 위한 메소드
sub, rsub	뺄셈(-)을 위한 메소드
mul, rmul	곱셈(*)을 위한 메소드
div, rdiv	나눗셈(/)을 위한 메소드
pow, rpow	거듭제곱(**)을 위한 메소드
floordiv, rfloordiv	소수점 내림(//)을 위한 메소드

```
import pandas as pd
import numpy as np

student1 = pd.Series({'국어':np.nan, '영어':80,'수학':90})
student2 = pd.Series({'수학':80, '국어':90})

print(student1)
print('₩n')
print(student2)
```

```
국어     NaN
영어    80.0
수학    90.0
dtype: float64


수학    80
국어    90
dtype: int64
```

```
add_data = student1.add(student2, fill_value=0)
sub_data = student1.sub(student2, fill_value=0)
mul_data = student1.mul(student2, fill_value=0)
div_data = student1.div(student2, fill_value=0)

result = pd.DataFrame([add_data,sub_data,mul_data,div_data], index = ['덧셈','뺄셈','곱셈','나눗셈'])
print(result)
```

```
       국어        수학    영어
덧셈   90.0   170.000  80.0
뺄셈  -90.0    10.000  80.0
곱셈    0.0  7200.000   0.0
나눗셈   0.0     1.125   inf
```

Series 연산

- index를 기준으로 연산

```
[4]  s1 = pd.Series([1, 2, 3, 4], index = ['a', 'b', 'c', 'd'])
```

```
[5]  s2 = pd.Series([10, 20, 30, 40], index = ['a', 'b', 'c', 'd'])
```

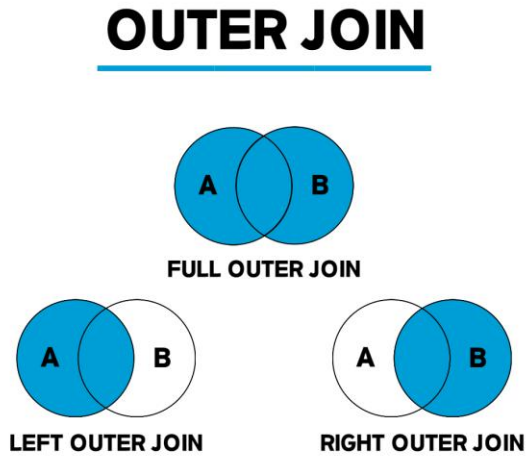
```
[6]  s1+s2
```

```
a    11
b    22
c    33
d    44
dtype: int64
```

Series 연산

- 짝이 맞지 않는 인덱스가 있는 경우, SQL의 outer join과 유사하게 동작

  → 결과 값에 두 인덱스의 값이 통합




```
[7]  s1 = pd.Series([1, 2, 3, 4], index = ['a', 'b', 'c', 'd'])
```

```
[8]  s3 = pd.Series([2, 2, 2, 2], index = ['b', 'c', 'd', 'e'])
```

```
[9]  # a, e
     s1+s3
```

```
a    NaN
b    4.0
c    5.0
d    6.0
e    NaN
dtype: float64
```

→ 단, 서로 짝이 맞지 않는 인덱스는 결과값이 NaN(Not-a-Number)을 의미하는 단어로 pandas에서는 누락된 값으로 취급)

Series 연산

짝이 맞지 않는 인덱스가 있는 경우, 결측치의 기본값을 지정해줄 수 있음

→ 메소드의 **fill_value** 인자를 이용하면 결측치를 채워줄 수 있음

코드 예제

```
s1.add(s3, fill_value=0)
```

실행 결과

```
a    1.0
b    4.0
c    5.0
d    6.0
e    2.0
dtype: float64
```

DataFrame

- index, column을 기준으로 연산

```
[16] d1
```

	c1	c2
id001	100.0	100.0
id002	100.0	100.0
id003	100.0	100.0

```
[17] d2
```

	c2	c3
id002	0	200
id003	400	600
id004	800	1000

```
d1 + d2
```

	c1	c2	c3
id001	NaN	NaN	NaN
id002	NaN	100.0	NaN
id003	NaN	500.0	NaN
id004	NaN	NaN	NaN

DataFrame

- index, column을 기준으로 연산

```
[16] d1
```

	c1	c2
id001	100.0	100.0
id002	100.0	100.0
id003	100.0	100.0

```
[17] d2
```

	c2	c3
id002	0	200
id003	400	600
id004	800	1000

```
[19] # (id001, c3), (id004, c1)
     d1.add(d2, fill_value=0)
```

	c1	c2	c3
id001	100.0	100.0	NaN
id002	100.0	100.0	200.0
id003	100.0	500.0	600.0
id004	NaN	800.0	1000.0

→ 둘다없는경우는NaN으로 채워짐

Series + DataFrame

- Series의 index(label)을 frame의 column으로 맞추고 아래 row로 전파

```
[29] frame
```

	a	b	c
0	1	1	1
1	1	1	1
2	1	1	1

```
[30] series
```

```
a    100
b    200
c    300
dtype: int64
```

```
frame + series
```

	a	b	c
0	101	201	301
1	101	201	301
2	101	201	301

Series + DataFrame

- 메소드를 사용하면, axis 인자로 축을 지정해 column으로 전파할 수 있음
  - axis = 0 또는 axis = 'index'

```
[29] frame
```

	a	b	c
0	1	1	1
1	1	1	1
2	1	1	1

```
series2
```

```
0    100
1    200
2    300
dtype: int64
```

```
frame.add(series2, axis=0)
```

	a	b	c
0	101	101	101
1	201	201	201
2	301	301	301

→ 단, index 이름이 일치해야함

데이터 필터링

조건에 맞는 데이터를 선택

```
import numpy as np
import pandas as pd

df = pd.DataFrame({
    'A' : ['foo','bar','foo','bar','foo','bar','foo','foo'],
    'B' : ['one','one','two','three','two','two','one','three'],
    'C' : np.random.randn(8),
    'D' : np.random.randn(8)
})

#'A'열이 'foo'인 행만 선택
print(df[df['A']=='foo'])
```

```
     A      B         C         D
0  foo    one  0.309534 -0.137455
2  foo    two  0.353697 -0.416813
4  foo    two  0.976033  0.257226
6  foo    one -0.266743  0.162457
7  foo  three -0.212963 -0.964870
```

비트 연산자

```
import pandas as pd
students = pd.DataFrame([
[3, '김영광','M'],
[13, '이은지', 'F'],
[23, '조유경','F'],
[4, '구재성', 'M'],
[14, '주재경', 'M'],
[24, '최지원', 'F']
], columns =['no', 'name', 'gender'])
print(students)
```

```
   no name gender
0   3  김영광      M
1  13  이은지      F
2  23  조유경      F
3   4  구재성      M
4  14  주재경      M
5  24  최지원      F
```

```
cond1 = students['no'] % 2 == 1 # no 컬럼의 값이 홀수이면
cond2 = students['gender'] == 'F' # gender 컬럼의 값이 'F'이면
subset = students[cond1 & cond2]  # and 를 사용하면 안된다
print(subset)
```

```
   no name gender
1  13  이은지      F
2  23  조유경      F
```

기술 통계 계산

메서드	설명
head,tail	series나 dataframe의 몇 개 데이터만 보여줌
describe	series나 dataframe의 각 컬럼에 대한 요약 통계
count	na 값을 제외한 값의 개수를 반환
min, max	최소값, 최대값
argmin, argmax	최소값, 최대값을 가진 색인의 위치(정수)를 반환
idxmin, idxmax	최소값, 최대값을 가진 색인의 값을 반환
sum	합
mean, median	평균, 중앙값
var, std	분산, 표준편차

기술 통계 계산

head / tail

- **head :** 위에서부터 일정한 개수만큼의 데이터만 보여줌 (default=5)
- **tail :** 아래에서부터 일정한 개수만큼의 데이터만 보여줌 (default=5)

`df.head(개수)`

`df.tail(개수)`

예제: stock.head()

```
stock.head()
```

	Date	kospi	kosdaq	gold_fut_132030	Bond_273130
0	2020. 1. 2 오후 3:30:00	2175.17	674.02	10845	108215
1	2020. 1. 3 오후 3:30:00	2176.46	669.93	11000	108565
2	2020. 1. 6 오후 3:30:00	2155.07	655.31	11245	108745
3	2020. 1. 7 오후 3:30:00	2175.54	663.44	11180	108400
4	2020. 1. 8 오후 3:30:00	2151.31	640.94	11360	108270

axis의 의미와 결과

Pandas의 집계 함수는 axis(축)를 기준으로 데이터를 따라 이동하며 계산하고, 그 결과를 반환합니다.

예제 DataFrame

```
import pandas as pd

df = pd.DataFrame({
    'A': [4, 7, 1],
    'B': [2, 5, 3],
    'C': [6, 8, 9]
}, index=['r1', 'r2', 'r3'])

df
```

열 방향 (axis=1) →

행 방향 (axis=0) ↓

	A	B	C
r1	4	2	6
r2	7	5	8
r3	1	3	9

핵심 정리

→ **axis=0 (기본값)**
: 행 방향으로 내려가며 계산 → 열별 결과 반환
즉, 각 열(컬럼)마다 하나의 값이 나옵니다.

→ **axis=1**
: 열 방향으로 이동하며 계산 → 행별 결과 반환
즉, 각 행마다 하나의 값이 나옵니다.

TIP "기준"이 아니라 "결과가 어떻게 반환되는지"를 함께 기억하세요!

axis=0 (기본값) → 열별 결과

```
df.max()  # axis=0 생략 (기본값)
```

행 방향으로 내려가며 최댓값 계산

각 열에서 가장 큰 값 선택

	A	B	C
r1	4	2	6
r2	7	5	8
r3	1	3	9

결과 (Series)

A	7
B	5
C	9

dtype: int64

열마다 하나의 값 → 열별 최댓값

axis=1 → 행별 결과

```
df.max(axis=1)
```

열 방향으로 이동하며 최댓값 계산

각 행에서 가장 큰 값 선택

	A	B	C
r1	4	2	6
r2	7	5	8
r3	1	3	9

결과 (Series)

r1	6
r2	8
r3	9

dtype: int64

행마다 하나의 값 → 행별 최댓값

더 쉽게 이해하기

- `df.sum(axis=0)` → 각 열의 합 — 열 방향(세로)으로 더하기 → 열별 합을 반환
- `df.sum(axis=1)` → 각 행의 합 — 행 방향(가로)으로 더하기 → 행별 합을 반환

df.sum(axis=0)

A	12	B	10	C	23

df.sum(axis=1)

r1	12	r2	20	r3	13

기술 통계 계산

- max/min/sum

```
[13] frame
```

	c1	c2	c3
i1	10.0	4.0	a
i2	10.0	2.5	b
i3	NaN	10.0	a

```
[9]  frame.max()
```

```
c1     10
c2     10
c3      b
dtype: object
```

→ 축을 지정하지 않으면 기본적으로 row을 기준으로 연산을 함

```
[10] frame.min(axis=1)
```

```
i1     4.0
i2     2.5
i3    10.0
dtype: float64
```

→ 컬럼을 기준으로 연산을 하고 싶으면 axis='columns' 또는 axis='1'을 인자로 넘겨주면 됨

기술 통계 계산

- unique
  - 중복되는 값을 제거하고 유일값만 담고 있는 Series를 반환

series.unique( )

```
obj
```

```
0    2.0
1    1.0
2    3.0
3    3.0
4    1.0
5    5.0
6    NaN
7    1.0
8    2.0
dtype: float64
```

```
[19] obj.unique()
```

```
array([ 2.,  1.,  3.,  5., nan])
```

→ 순서를 정렬해서 반환하지 않음

기술 통계 계산

- value_counts
  - 값을 인덱스(라벨)로 하고 그 값의 개수를 담고 있는 Series를 반환

```
obj
```

```
0    2.0
1    1.0
2    3.0
3    3.0
4    1.0
5    5.0
6    NaN
7    1.0
8    2.0
dtype: float64
```

```
[20] obj.value_counts()
```

```
1.0    3
3.0    2
2.0    2
5.0    1
dtype: int64
```

→ 값에 대해 내림차순으로 정렬하여 반환

```
# normalize
obj.value_counts(normalize=True)
```

```
1.0    0.375
3.0    0.250
2.0    0.250
5.0    0.125
dtype: float64
```

Sort_index

- index를 기준으로 정렬

```
series.sort_index(axis=0, ascending=True, na_position='last')
```

- axis : 정렬 기준 축, 이전과 미찬가지로 0('index')과 1('column')가 있음
- ascending : True이면 오름차순, False이면 내림차순으로 정렬
- na_position : 정렬 시 NaN값의 위치 지정, {'first', 'last'}

1 obj

```
obj
a    1
d    2
e    3
b   -1
c   -2
dtype: int64
```

2 obj.sort_index()

```
obj.sort_index()
a    1
b   -1
c   -2
d    2
e    3
dtype: int64
```

3 obj.sort_index(ascending=False)

```
obj.sort_index(ascending=False)
e    3
d    2
c   -2
b   -1
a    1
dtype: int64
```

Sort_values

- 값을 기준으로 정렬

```
series.sort_values(axis=0, ascending=True, na_position='last')
```

- axis : 정렬 기준 축, 이전과미찬가지로 0('index')과 1('column')가 있음
- ascending : True이면 오름차순, False이면 내림차순으로 정렬
- na_position : 정렬시 NaN값의 위치 지정, {'first', 'last'}

1 obj

obj

0	10.0
1	NaN
2	20.0
3	0.0
4	NaN

dtype: float64

2 obj.sort_values()

obj.sort_values()

3	0.0
0	10.0
2	20.0
1	NaN
4	NaN

dtype: float64

3 obj.sort_values(ascending=False)

obj.sort_values(ascending=False)

2	20.0
0	10.0
3	0.0
1	NaN
4	NaN

dtype: float64

Sort_index

- **index를 기준으로 정렬**

```
DataFrame.sort_index(by, axis=0, ascending=True, na_position='last')
```

- by : 여러개의 컬럼이나 인덱스를 정렬할 때 유용한 인자로, 정렬 기준
- axis : 정렬 기준 축, 이전과 마찬가지로 0('index')과 1('column')가 있음
- ascending : True이면 오름차순, False이면 내림차순으로 정렬
- na_position : 정렬시 NaN값의 위치 지정, {'first', 'last'}

→ by에서 정해준 값을 기준으로 정렬

1 `[16] frame`

	e	d	f
a	0	1	2
c	3	4	5
b	6	7	8

2 `[17] frame.sort_index()`

	e	d	f
a	0	1	2
b	6	7	8
c	3	4	5

기본값 : axis=0, ascending='True, na_position='last'

3 `[18] frame.sort_index(axis=1)`

	d	e	f
a	1	0	2
c	4	3	5
b	7	6	8

axis=1 : column 방향으로 정렬

4 `[19] frame.sort_valuee(by='a', ascending=False)`

	a	b
2	10	3
3	10	1
4	8	4
0	5	2
1	4	0

by='a' 열의 값을 기준으로 내림차순 정렬

Sort_index

- df.sort_index
  - index를 기준으로 정렬

```
[21] frame
```

	a	b
0	5	2
1	4	0
2	10	3
3	10	1
4	8	4

```
frame.sort_values(by=['a', 'b'], ascending = [False, True])
```

	a	b
3	10	1
2	10	3
4	8	4
0	5	2
1	4	0

→ by에 여러 개의 기준을 주고 싶으면 리스트로 인자를 넘김.
리스트에 있는 순서가 우선순위가 되며, 이에 상응하여
ascending인자도 multi-value를 줄 수 있음

Series_map

map은 series의 각각의 element들을 다른 어떤 값으로 대체하는 역할

```
series.map(arg, na_action=None)
```

- arg : 대체할 값으로 function, dictionary, series 등
- na_action : NaN값이 존재할 경우 어떻게 작동할지에 대한 인자

{None, 'ignore'} None이라면 NaN에도 작동, 'ignore'은 NaN은 무시

[28] series

0	100
1	200
2	300

dtype: int64



[29] series.map({100: 'C', 200: 'B', 300: 'A'})

0	C
1	B
2	A

dtype: object

arg가 dict인 경우, 원본 series에 있는 값을 key로 바꿔 줄값을 value로 설정

만일, 원본 데이터에 있는 값이 dict에 빠졌을 경우에는 NaN으로 대체

Series_map

series.**map**(**arg**, **na_action**=*None*)

```
[28] series

0    100
1    200
2    300
dtype: int64
```

→ arg에 함수도 가능

```
[25] series.map('${}'.format)

0    $100
1    $200
2    $300
dtype: object
```

```
[26] series.map('{}달러'.format)

0    100달러
1    200달러
2    300달러
dtype: object
```

```
# lambda
f = lambda x: np.add(x, 3)
series.map(f)

0    103
1    203
2    303
dtype: int64
```

Series_apply

어떠한 함수를 series 의 각각의 element에 적용시켜주는 함수

map함수 보다 적용할 수 있는 함수의 범위가 넓음

```
series.apply(func, args, **kwargs)
```

- func : series에 적용될 함수
- args : series를 제외한 함수에 들어갈 다른 매개변수
- **kwargs : 함수에 넘겨줄 키워드 인자들

[9]
```
s
```

```
London      20
New York    21
Helsinki    12
dtype: int64
```

[11]
```
def sub_custom_value(x, val):
    return x - val
```

[13]
```
s.apply(sub_custom_value, args=(10,))
```

```
London      10
New York    11
Helsinki     2
dtype: int64
```

DataFrame_apply

축별로 함수를 적용

df.apply(func, axis)

- func : df에 적용될 함수
- axis : 함수가 적용될 기준 축

```
[18] frame
```

	a	b	c	d
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11

```
[22] frame.apply(lambda x: x.max()-x.min())
```

```
a    8
b    8
c    8
d    8
dtype: int64
```

```
[23] frame.apply(lambda x: x.max()-x.min(), axis=1)
```

```
0    3
1    3
2    3
dtype: int64
```

DataFrame_applymap

모든 원소에 원소별로 함수를 적용

df.applymap(func)

```
[18] frame
```

	a	b	c	d
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11

```
[24] frame.applymap(lambda x: x**2)
```

	a	b	c	d
0	0	1	4	9
1	16	25	36	49
2	64	81	100	121

데이터 삭제_drop

row나 column에서 특정한 label을 삭제하는 함수

```
[31] frame
```

	c1	c2	c3	c4
r1	0	1	2	3
r2	4	5	6	7
r3	8	9	10	11
r4	12	13	14	15

```
[32] frame.drop('r1')
```

	c1	c2	c3	c4
r2	4	5	6	7
r3	8	9	10	11
r4	12	13	14	15

→ 기본 axis = 0

```
frame.drop('c2', axis = 1)
```

	c1	c3	c4
r1	0	2	3
r2	4	6	7
r3	8	10	11
r4	12	14	15

→ axis를 명시해서 column을 삭제할 수도 있음

데이터 삭제_drop

row나 column에서 특정한 label을 삭제하는 함수

```
[31] frame
```

	c1	c3	c4
r1	0	2	3
r2	4	6	7
r3	8	10	11
r4	12	14	15

```
[34] frame.drop(columns=['c3', 'c4'])
```

	c1	c2
r1	0	1
r2	4	5
r3	8	9
r4	12	13

→ columns라는 인자에 리스트로 label 값을 넘기면 여러 개의 label을 한번에 지울 수 있음(axis 설정이 따로 필요 없음)

데이터 삭제_drop

row나 column에서 특정한 label을 삭제하는 함수

```
[31] frame
```

	c1	c2	c3	c4
r1	0	1	2	3
r2	4	5	6	7
r3	8	9	10	11
r4	12	13	14	15

```
[35] # inplace
     frame.drop(['r2'], inplace=True)
     frame
```

	c1	c2	c3	c4
r1	0	1	2	3
r3	8	9	10	11
r4	12	13	14	15

→ inplace 인자를 이용할 수 있음

```
# 할당
frame = frame.drop(['r3'])
frame
```

	c1	c2	c3	c4
r1	0	1	2	3
r4	12	13	14	15

→ inplace 인자를 이용하지 않고, 직접 할당

데이터 병합 - concat

기준이 되는 축을 따라 객체를 이어 붙이는 함수

```
pd.concat(objs, axis, join='outer')
```

- axis : 기준이 되는 축
- join : 어떠한 방식으로 이어 붙일 지 {'outer', 'inner'}

s1

```
c    100
b    200
dtype: int64
```

s2

```
c    300
d    300
e    300
dtype: int64
```

s3

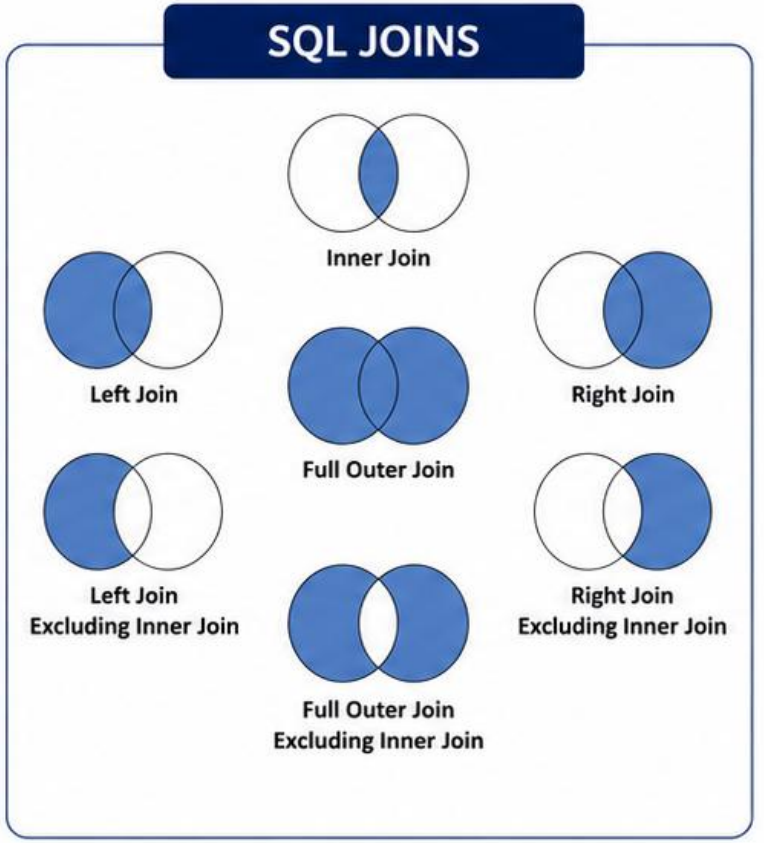
```
f    500
g    600
dtype: int64
```

```
[46] print(s1, s2, s3, sep='\n\n')
```

```
[38] pd.concat([s1, s2, s3])

c    100
b    200
c    300
d    300
e    300
f    500
g    600
dtype: int64
```

axis=0 을따라 쪽이어붙이기



axis 0 을따라 쪽 이어붙이게 됨

데이터 병합 - concat

pd.concat(objs, **axis**, **join=***'outer'***)**

```
[46] print(s1, s2, s3, sep='\n\n')
```

```
c    100
b    200
dtype: int64

c    300
d    300
e    300
dtype: int64

f    500
g    600
dtype: int64
```

```
pd.concat([s1, s2], axis=1)
```

	0	1
c	100.0	300.0
b	200.0	NaN
d	NaN	300.0
e	NaN	300.0

→ axis=1을 따라 쭉 이어붙이게 됨.

즉, 새로운 컬럼이 생기고 dataframe이 됨

데이터 병합 - concat

```
[47] data1
```

	id	col1	col2
0	01	43	1918
1	02	17	1103
2	03	31	1269
3	04	12	1991
4	05	19	1853
5	06	1	1127

```
data2
```

	id	col1
0	04	3876
1	05	2076
2	06	1509
3	07	4533

```
[49] pd.concat([data1, data2])
```

	id	col1	col2
0	01	43	1918.0
1	02	17	1103.0
2	03	31	1269.0
3	04	12	1991.0
4	05	19	1853.0
5	06	1	1127.0
0	04	3876	NaN
1	05	2076	NaN
2	06	1509	NaN
3	07	4533	NaN

데이터 병합 - concat

```
[47] data1
```

	id	col1	col2
0	01	43	1918
1	02	17	1103
2	03	31	1269
3	04	12	1991
4	05	19	1853
5	06	1	1127

```
data2
```

	id	col1
0	04	3876
1	05	2076
2	06	1509
3	07	4533

```
pd.concat([data1, data2], axis=1)
```

	id	col1	col2	id	col1
0	01	43	1918	04	3876.0
1	02	17	1103	05	2076.0
2	03	31	1269	06	1509.0
3	04	12	1991	07	4533.0
4	05	19	1853	NaN	NaN
5	06	1	1127	NaN	NaN

데이터 병합 - merge

Key를 이용해 데이터의 row를 기준으로 연결시켜 합침. (sql의 join과 유사)

```
pd.merge(objs, how='inner', on)
```

- objs : 병합할 자료
- how : 어떤 방식으로 병합할 것 인가 {'inner', 'outer', 'left', 'right'}
- on : 어떤 label을 기준으로 병합할 것 인가, 기본적으로 중복되는 기준으로 병합을 함

1 data1

	id	col1	col2
0	01	5	1083
1	02	29	1548
2	03	6	1594
3	04	16	1763
4	05	7	1305
5	06	20	1070

2 data2

	id	col1
0	04	4532
1	05	2599
2	06	4556
3	07	3547

3 # inner join

```
pd.merge(data1, data2, on='id')
```

	id	col1_x	col2	col1_y
0	04	16	1763	4532
1	05	7	1305	2599
2	06	20	1070	4556

기본값 : how='inner'
→ 두 데이터에서 모두 존재하는 key만 결과에 포함

Missing data

함수	설명
isnull	누락되거나 NA(not available) 값을 알려주는 불리언 값들이 저장된 객체를 반환
notnull	isnull과 반대되는 메서드
fillna	누락된 데이터에 값을 채우는 메서드. (특정한 값이나 ffill, bfill 같은 보간 메서드 적용)
dropna	누락된 데이터가 있는 축(로우, 컬럼)을 제외시키는 메서드

Missing data

isnull(), isna()

- 누락되거나 NAN인 값을 알려주는 불리언값이 저장된 같은 형의 객체를 반환
- None, np.NAN

```
import pandas as pd
import numpy as np

# (슬라이드와 같은 예시: Series)
obj = pd.Series(["apple", "mango", np.nan, None, "peach"])
print(obj)
```

```
0    apple
1    mango
2      NaN
3     None
4    peach
dtype: object
```

obj.isnull()

	0
0	False
1	False
2	True
3	True
4	False

dtype: bool

obj.isna()

	0
0	False
1	False
2	True
3	True
4	False

dtype: bool

obj.isnull().sum()

np.int64(2)

obj.isna().sum()

np.int64(2)

Missing data_drop



dropna(axis=0, how='any', thresh)

💡 누락된 데이터가 있는 축(컬럼, 로우)을 제거하는 메서드

axis	기준이 되는 축	0 이면 row, 1 이면 columns
how	NA를 찾았을 때	어떤 방식으로 찾아서 지울 지에 대한 인자 { 'any', 'all' }
thresh	지우는 기준	NA value를 몇 개까지 허용할지에 대한 인자

[17] obj

```
0     apple
1     mango
2       NaN
3      None
4     peach
dtype: object
```

[18] obj.dropna()

```
0     apple
1     mango
4     peach
dtype: object
```

[5] frame

	x1	x2	x3	y
0	NaN	NaN	NaN	NaN
1	10.0	5.0	40.0	6.0
2	5.0	2.0	30.0	8.0
3	20.0	NaN	20.0	6.0
4	15.0	3.0	10.0	NaN

[6] frame.dropna()

	x1	x2	x3	y
1	10.0	5.0	40.0	6.0
2	5.0	2.0	30.0	8.0

Missing data_drop

dropna(axis=*0*, how=*'any'*, thresh)

```
[5] frame
```

	x1	x2	x3	y
0	NaN	NaN	NaN	NaN
1	10.0	5.0	40.0	6.0
2	5.0	2.0	30.0	8.0
3	20.0	NaN	20.0	6.0
4	15.0	3.0	10.0	NaN

```
# how
frame.dropna(how='all')
```

	x1	x2	x3	y
1	10.0	5.0	40.0	6.0
2	5.0	2.0	30.0	8.0
3	20.0	NaN	20.0	6.0
4	15.0	3.0	10.0	NaN

```
frame.dropna(thresh=2)
```

	x1	x2	x3	x4
1	10.0	5.0	40.0	6.0
2	5.0	2.0	30.0	8.0
3	20.0	NaN	20.0	6.0
4	15.0	3.0	10.0	NaN

```
frame['e'] = np.nan
frame
```

	x1	x2	x3	y	e
0	NaN	NaN	NaN	NaN	NaN
1	10.0	5.0	40.0	6.0	NaN
2	5.0	2.0	30.0	8.0	NaN
3	20.0	NaN	20.0	6.0	NaN
4	15.0	3.0	10.0	NaN	NaN

Missing data_fillna

fillna(objs, method='none')

- 누락된 데이터를 대신할 값으로 채우거나, 'ffill'이나 'bfill' 같은 보간 메서드를 적용
- Method: 보간방법 {'ffill', 'bfill', 'backfill', 'pad', None}

[16] `frame`

	x1	x2	x3	y	e
0	NaN	NaN	NaN	NaN	NaN
1	10.0	5.0	40.0	6.0	NaN
2	5.0	2.0	30.0	8.0	NaN
3	20.0	NaN	20.0	6.0	NaN
4	15.0	3.0	10.0	NaN	NaN

`frame.fillna(0)`

	x1	x2	x3	y	e
0	0.0	0.0	0.0	0.0	0.0
1	10.0	5.0	40.0	6.0	0.0
2	5.0	2.0	30.0	8.0	0.0
3	20.0	0.0	20.0	6.0	0.0
4	15.0	3.0	10.0	0.0	0.0

`frame.fillna({'x1': 10, 'y': 0})`

	x1	x2	x3	y	e
0	10.0	NaN	NaN	0.0	NaN
1	10.0	5.0	40.0	6.0	NaN
2	5.0	2.0	30.0	8.0	NaN
3	20.0	NaN	20.0	6.0	NaN
4	15.0	3.0	10.0	0.0	NaN

컬럼 이름을 키로, 대체하고자 하는 값을 value로 하는 dict를 넘겨주면 각 컬럼에 매칭하여 값을 변경

데이터 변형_중복 제거

1. duplicated(): 각 row가 중복인지(True)인지 (False)인지를 알려주는 boolean series 반환
2. drop_duplicates(): duplicate를 적용한 결과가 False인 것들만 모아서 dataframe반환

[22] data

	id	name	phone
0	0001	a	0
1	0002	b	1
2	0003	c	2
3	0001	a	3

[18] data.duplicated()

0	False
1	False
2	False
3	True

dtype: bool

data.drop_duplicates()

	id	name
0	0001	a
1	0002	b
2	0003	c

data.drop_duplicates(subset=['id'], keep='last')

	id	name	phone
1	0002	b	1
2	0003	c	2
3	0001	a	3

→ subset : 중복을 검색할 기준
keep : 중복 자료에서 어떤 것을 남길지를 파악하는 인자

데이터 변형_치환

- replace : 특정값을 치환하는 함수

```
[23] obj = pd.Series([10, -999, 4, 5, 7, 'n'])
```

```
[24] obj.replace(-999, np.nan)
```

```
0     10
1    NaN
2      4
3      5
4      7
5      n
dtype: object
```

```
obj.replace([-999, 'n'], np.nan)
```

```
0    10.0
1     NaN
2     4.0
3     5.0
4     7.0
5     NaN
dtype: float64
```

→ 한 개 이상의 값도 치환 가능

데이터 변형_binning

연속형 데이터를 일정한 구간(구간값)으로 나눠 각 데이터 값을 해당 구간의 범주로 변환하는 과정

- cut

```
[26] ages = [20, 35, 67, 39, 59, 44, 56, 77, 28, 20, 22, 80, 32, 46, 52, 19, 33, 5, 15, 50, 29, 21, 33, 48, 85, 80, 31, 10]
```

```
[27] bins = [0, 20, 40, 60, 100]
```

```
[28] cuts = pd.cut(ages, bins)
     cuts
```

```
[(0, 20], (20, 40], (60, 100], (20, 40], (40, 60], ..., (40, 60], (60, 100], (60, 100], (20, 40], (0, 20]]
Length: 28
Categories (4, interval[int64]): [(0, 20] < (20, 40] < (40, 60] < (60, 100]]
```

- cut 메소드의 결과는 Categorical이라는 특수한 객체

```
[29] cuts.categories
```

```
IntervalIndex([(0, 20], (20, 40], (40, 60], (60, 100]],
              closed='right',
              dtype='interval[int64]')
```

```
[30] cuts.codes
```

```
array([0, 1, 3, 1, 2, 2, 2, 3, 1, 0, 1, 3, 1, 2, 2, 0, 1, 0, 0, 2, 1, 1,
       1, 2, 3, 3, 1, 0], dtype=int8)
```

```
[36] # 구간을 균등한 길이로 나눔
     pd.cut(ages, 4, precision=1).value_counts()
```

```
(4.9, 25.0]     8
(25.0, 45.0]    9
(45.0, 65.0]    6
(65.0, 85.0]    5
dtype: int64
```

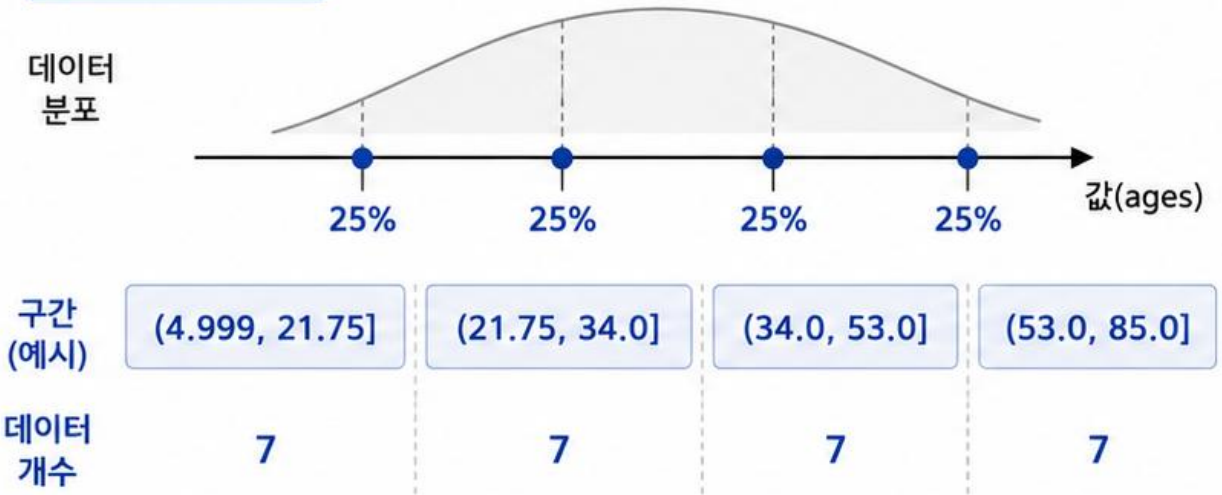
데이터 변형_binning

연속형 데이터를 일정한 **구간(구간값)**으로 나눠 각 데이터 값을 해당 구간의 범주로 변환하는 과정

- **qcut**(quantile cut)
  - 분위수 기준으로 나눠 각 구간의 데이터 개수가 (거의) 같도록 binning

cut	구간 폭 동일(equal-width)
qcut	구간 개수 동일(equal-frequency)

qcut(예: 4분위)



구간 (예시)	(4.999, 21.75]	(21.75, 34.0]	(34.0, 53.0]	(53.0, 85.0]
데이터 개수	7	7	7	7

[34]
```
# 개수들이 균등한 비율이 되도록 나눔
pd.qcut(ages, 4).value_counts()
```

```
(4.999, 21.75]    7
(21.75, 34.0]     7
(34.0,  53.0]     7
(53.0,  85.0]     7
dtype: int64
```

4분위로 나눴더니 각 구간에 관측치가 7개씩 배정됨.

데이터 변형_ get_dummies

- 결과 컬럼 수 = 범주 개수(고유값 수)
- 각 행에서 해당 범주만 1, 나머지는 0

No	day
1	월
2	화
3	수



No	월	화	수
1	1	0	0
2	0	1	0
3	0	0	1

```
[37] df = pd.DataFrame({'col1': [10, 20, 30],
                        'col2': ['a', 'b', 'a']})
     df
```

	col1	col2
0	10	a
1	20	b
2	30	a

```
pd.get_dummies(df)
```

	col1	col2_a	col2_b
0	10	1	0
1	20	0	1
2	30	1	0

데이터 변형_ get_dummies

- 결과 컬럼 수 = 범주 개수(고유값 수)
- 각 행에서 해당 범주만 1, 나머지는 0

```
df = pd.DataFrame({'col1': ['001', '002', '003', '004', '005', '006'],
                   'col2': [10, 20, 30, 40, 50, 60],
                   'col3': ['서울시', '경기도', '서울시', '제주도', '경기도', '서울시']})
df
```

	col1	col2	col3
0	001	10	서울시
1	002	20	경기도
2	003	30	서울시
3	004	40	제주도
4	005	50	경기도
5	006	60	서울시

```
pd.get_dummies(df)
```

	col2	col1_001	col1_002	col1_003	col1_004	col1_005	col1_006	col3_경기도	col3_서울시	col3_제주도
0	10	1	0	0	0	0	0	0	1	0
1	20	0	1	0	0	0	0	1	0	0
2	30	0	0	1	0	0	0	0	1	0
3	40	0	0	0	1	0	0	0	0	1
4	50	0	0	0	0	1	0	1	0	0
5	60	0	0	0	0	0	1	0	1	0

groupby

- 그룹연산은 분리-적용-결합의 과정

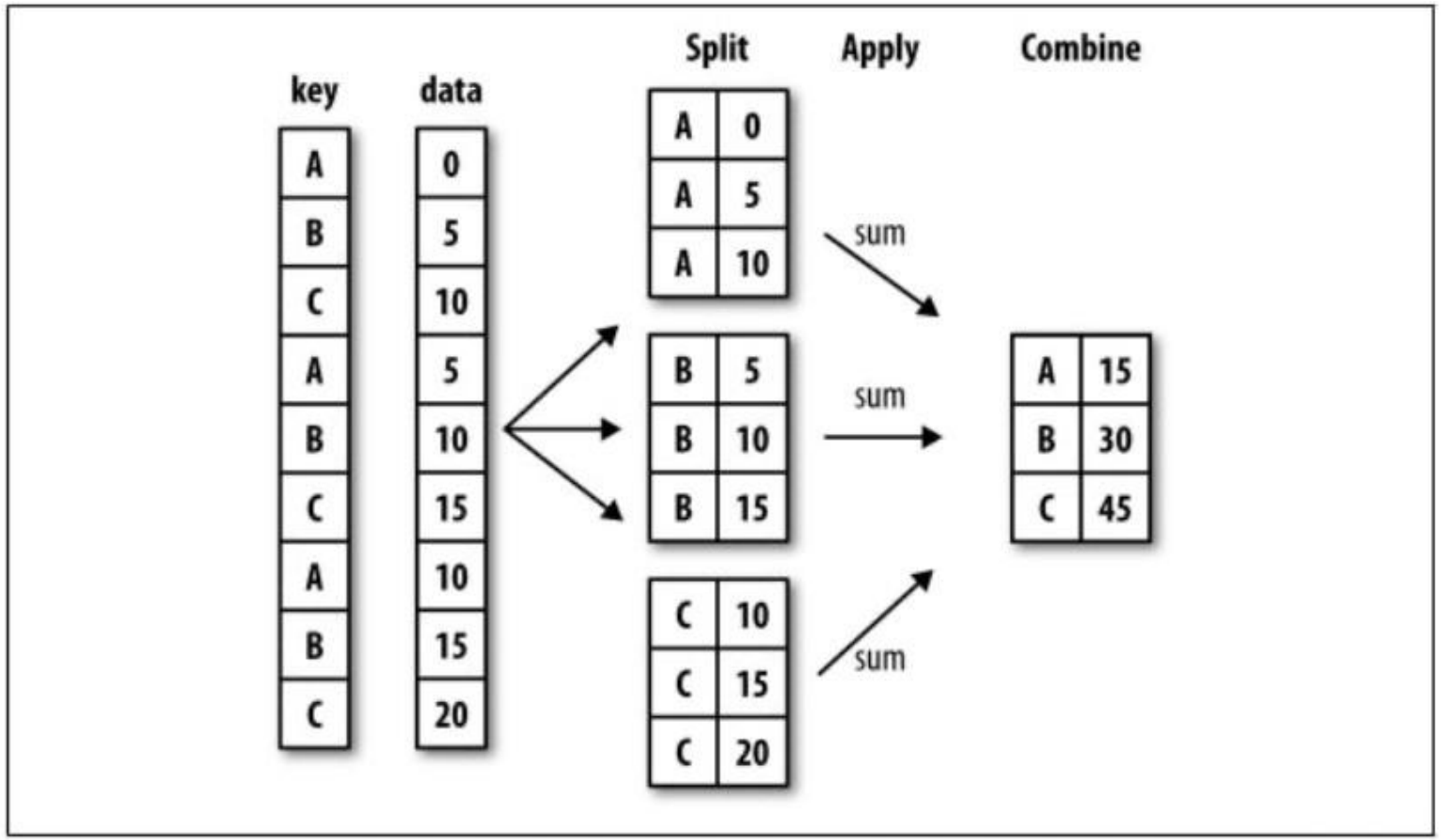


Figure 10-1. Illustration of a group aggregation

groupby

```
[55] kbo = pd.read_csv('kbo.csv')
     kbo.head()
```

	연도	순위	팀	경기수	승	패	무	승률	게임차
0	2019	1	두산	144	88	55	1	0.615	0.0
1	2019	2	키움	144	86	57	1	0.601	2.0
2	2019	3	SK	144	88	55	1	0.615	0.0
3	2019	4	LG	144	79	64	1	0.552	9.0
4	2019	5	NC	144	73	69	2	0.514	14.5

```
kbo.shape
```

```
(30, 9)
```

```
kbo.columns
```

```
Index(['연도', '순위', '팀', '경기수', '승', '패', '무', '승률', '게임차'], dtype
='object')
```

```
kbo['팀'].unique()
```

```
array(['두산', '키움', 'SK', 'LG', 'NC', 'KT', 'KIA', '삼성', '한화', '롯데', '넥센'],
      dtype=object)
```

groupby

.info() 함수는 데이터에 대한 전반적인 정보를 나타냅니다.

df를 구성하는 행과 열의 크기, 컬럼명, 컬럼을 구성하는 값의 자료형 등을 출력함

```
kbo.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 9 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   연도      30 non-null     int64  
 1   순위      30 non-null     int64  
 2   팀       30 non-null     object 
 3   경기수     30 non-null     int64  
 4   승       30 non-null     int64  
 5   패       30 non-null     int64  
 6   무       30 non-null     int64  
 7   승률      30 non-null     float64
 8   게임차     30 non-null     float64
dtypes: float64(2), int64(6), object(1)
memory usage: 2.2+ KB
```

groupby

.describe() 수치형 변수들만 기준으로, count, 평균, 표준편차, 최소/최댓값, 4분위를 기준으로 25%, 50%

75%에 해당하는 값들을 테이블로 출력해줌

```
kbo.describe()
```

	연도	순위	경기수	승	패	무	승률	게임차
count	30.000000	30.000000	30.0	30.000000	30.000000	30.000000	30.000000	30.000000
mean	2018.000000	5.466667	144.0	71.200000	71.200000	1.600000	0.499800	12.833333
std	0.830455	2.921187	0.0	11.871147	11.591912	1.275769	0.082079	12.576204
min	2017.000000	1.000000	144.0	48.000000	51.000000	0.000000	0.340000	-14.500000
25%	2017.000000	3.000000	144.0	61.250000	62.500000	1.000000	0.431750	2.375000
50%	2018.000000	5.000000	144.0	70.500000	71.500000	1.000000	0.494500	9.750000
75%	2019.000000	8.000000	144.0	79.000000	80.750000	2.000000	0.558000	19.500000
max	2019.000000	10.000000	144.0	93.000000	94.000000	5.000000	0.646000	39.000000

groupby

.describe() 수치형 변수들만 기준으로, count, 평균, 표준편차, 최소/최댓값, 4분위를 기준으로 25%, 50%

75%에 해당하는 값들을 테이블로 출력해줌

```
kbo.describe()
```

	연도	순위	경기수	승	패	무	승률	게임차
count	30.000000	30.000000	30.0	30.000000	30.000000	30.000000	30.000000	30.000000
mean	2018.000000	5.466667	144.0	71.200000	71.200000	1.600000	0.499800	12.833333
std	0.830455	2.921187	0.0	11.871147	11.591912	1.275769	0.082079	12.576204
min	2017.000000	1.000000	144.0	48.000000	51.000000	0.000000	0.340000	-14.500000
25%	2017.000000	3.000000	144.0	61.250000	62.500000	1.000000	0.431750	2.375000
50%	2018.000000	5.000000	144.0	70.500000	71.500000	1.000000	0.494500	9.750000
75%	2019.000000	8.000000	144.0	79.000000	80.750000	2.000000	0.558000	19.500000
max	2019.000000	10.000000	144.0	93.000000	94.000000	5.000000	0.646000	39.000000

```
kbo.isnull().sum()
# kbo.isna().sum()
```

```
연도      0
순위      0
팀       0
경기수     0
승       0
패       0
무       0
승률      0
게임차     0
dtype: int64
```

groupby

```
kbo.groupby('팀')
```

```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x7fb8d4dcb7b8>
```

→ groupby를 하면 group으로 묶인 GroupBy 객체를반환.

이객체는그룹연산을위해필요 한모든정보를가지고 있음

최적화된 groupby 메소드

메소드	설명
count	그룹에서 NA가 아닌 값의 수를 반환
sum	NA가 아닌 값들의 합
mean	NA가 아닌 값들의 평균
median	NA가 아닌 값들의 산술 중간값
std, var	편향되지 않은(n-1을 분모로 하는) 표준편차, 분산
min, max	NA가 아닌 값들 중 최소값과 최대값
prod	NA가 아닌 값들의 곱
first, last	NA가 아닌 값들 중 첫째 값과 마지막 값

groupby

최적화 된 groupby 메소드

```
kbo.groupby('팀').count()
```

팀	연도	순위	경기수	승	패	무	승률	게임차
KIA	3	3	3	3	3	3	3	3
KT	3	3	3	3	3	3	3	3
LG	3	3	3	3	3	3	3	3
NC	3	3	3	3	3	3	3	3
SK	3	3	3	3	3	3	3	3
넥센	2	2	2	2	2	2	2	2
두산	3	3	3	3	3	3	3	3
롯데	3	3	3	3	3	3	3	3
삼성	3	3	3	3	3	3	3	3
키움	1	1	1	1	1	1	1	1
한화	3	3	3	3	3	3	3	3

→'팀' 컬럼에 있었던 unique 값을 기준으로 groupby함, 새롭게 생성된 색인

groupby

- 두개이상의 색인으로도 groupby 할수있음
  → 인자로 넘기는 순서에 대응하여
  계층적 인덱스를 만들어냄

```
kbo.groupby(['연도','팀']).sum()
```

연도	팀	순위	경기수	승	패	무	승률	게임차
2017	KIA	1	144	87	56	1	0.608	0.0
	KT	10	144	50	94	0	0.347	37.5
	LG	6	144	69	72	3	0.489	17.0
	NC	4	144	79	62	3	0.560	7.0
	SK	5	144	75	68	1	0.524	12.0
	넥센	7	144	69	73	2	0.486	17.5
	두산	2	144	84	57	3	0.596	2.0
	롯데	3	144	80	62	2	0.563	6.5
	삼성	9	144	55	84	5	0.396	30.0
	한화	8	144	61	81	2	0.430	25.5
2018	KIA	5	144	70	74	0	0.486	8.5
	KT	9	144	59	82	3	0.418	18.0

groupby

- 컬럼이나 컬럼의 일부만 선택하려면?

  컬럼 이름이 담긴 배열 이용

```
kbo.groupby('팀')['승률'].max()
```

```
팀
KIA     0.608
KT      0.500
LG      0.552
NC      0.560
SK      0.615
넥센      0.521
두산      0.646
롯데      0.563
삼성      0.486
키움      0.601
한화      0.535
Name: 승률, dtype: float64
```

```
kbo.groupby(['연도','팀'])[['승률','순위']].ma
```

연도	팀	승률	순위
2017	KIA	0.608	1
	KT	0.347	10
	LG	0.489	6
	NC	0.560	4
	SK	0.524	5
	넥센	0.486	7
	두산	0.596	2
	롯데	0.563	3
	삼성	0.396	9
	한화	0.430	8

groupby

- groupby.get_group

```
grouped= kbo.groupby('팀')
type(grouped)
```

```
pandas.core.groupby.generic.DataFrameGroupBy
```

```
for name, group in grouped:
    print(name)
    print(group)

    print('-'*50)
```

```
KIA
      연도  순위   팀  경기수  승  패  무    승률  게임차
6   2019   7  KIA  144  62  80  2  0.437  25.5
14  2018   5  KIA  144  70  74  0  0.486   8.5
20  2017   1  KIA  144  87  56  1  0.608   0.0
--------------------------------------------------
KT
      연도  순위  팀  경기수  승  패  무    승률  게임차
5   2019   6  KT  144  71  71  2  0.500  16.5
18  2018   9  KT  144  59  82  3  0.418  18.0
29  2017  10  KT  144  50  94  0  0.347  37.5
```

→ groupby를 하면
group으로묶인 GroupBy 객체를반환

```
grouped.get_group('한화')
```

	연도	순위	팀	경기수	승	패	무	승률	게임차
8	2019	9	한화	144	58	86	0	0.403	30.5
12	2018	3	한화	144	77	67	0	0.535	1.5
27	2017	8	한화	144	61	81	2	0.430	25.5

groupby

- groupby.agg
  - 그룹별로 특정한 집계함수를 적용

```
grouped['순위'].agg([('함수', lambda val : val.max()-val.min())])
```

팀	함수
KIA	6
KT	4
LG	4
NC	6
SK	3

→ 튜플을 이용하면 적용되는 함수의 이름을 따로 지정해줄 수 있음

groupby

- groupby.agg
  - 그룹별로 특정한 집계함수를 적용

```
grouped.agg({'순위':np.mean, '승률':[np.mean, np.std]})
```

	순위	승률	
	mean	mean	std
팀			
KIA	4.333333	0.510333	0.088059
KT	8.333333	0.421667	0.076566
LG	6.000000	0.505667	0.040649
NC	6.333333	0.493333	0.079053
SK	3.333333	0.561333	0.047648

→ 딕셔너리를 이용하면 컬럼별로 다른함수를 적용 할수있음

groupby

- groupby.filter
  - 그룹화 한 후 필터링

```
[15] kbo.groupby('팀').filter(lambda x: len(x)==2)
```

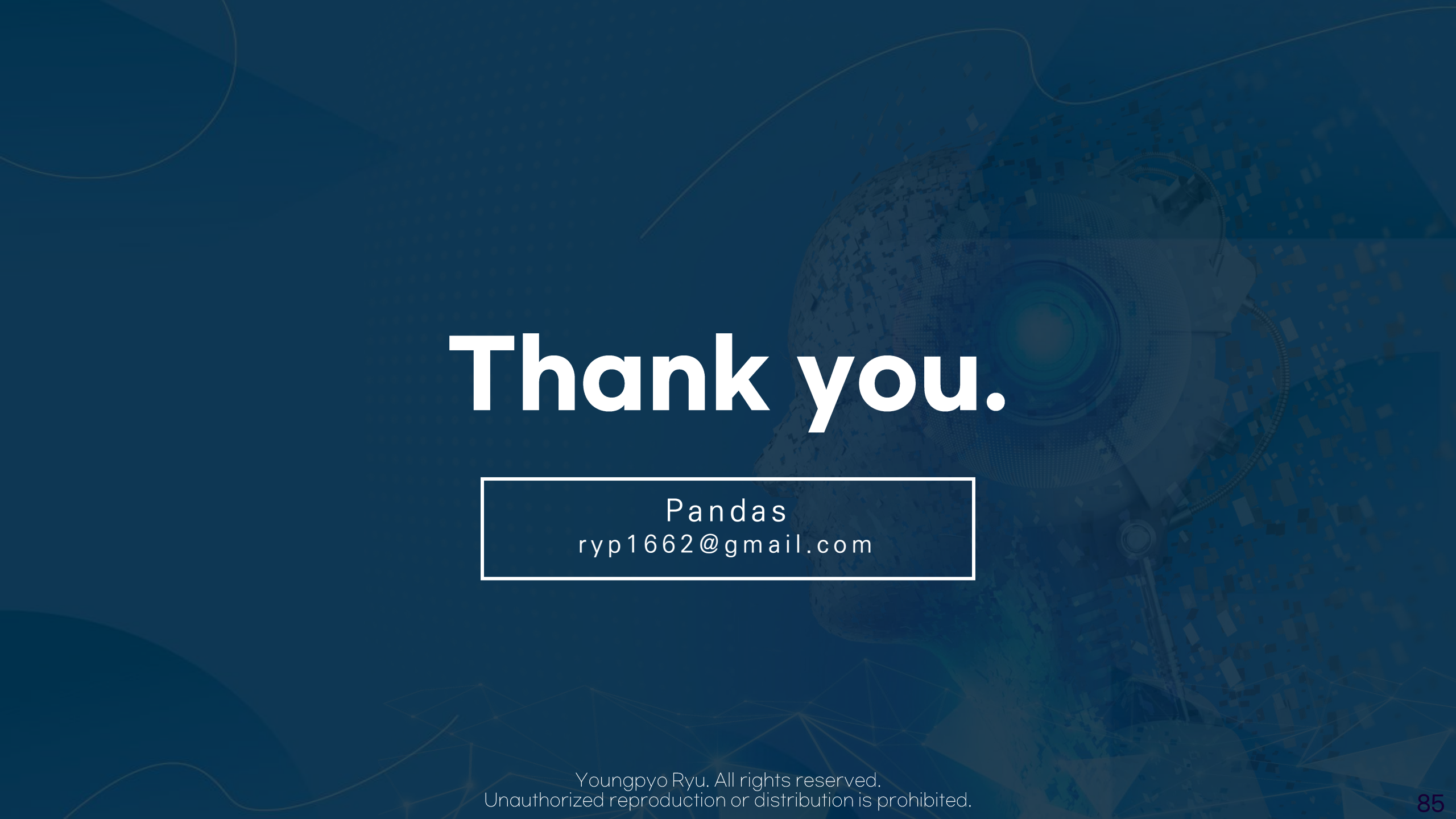
	연도	순위	팀	경기수	승	패	무	승률	게임차
13	2018	4	넥센	144	75	69	0	0.521	3.5
26	2017	7	넥센	144	69	73	2	0.486	17.5

groupby

- groupby.filter
  - 그룹화 한 후 필터링

```
kbo.groupby('팀').filter(lambda x: x['순위'].min()==1)
```

	연도	순위	팀	경기수	승	패	무	승률	게임차
0	2019	1	두산	144	88	55	1	0.615	0.0
6	2019	7	KIA	144	62	80	2	0.437	25.5
10	2018	1	두산	144	93	51	0	0.646	-14.5
14	2018	5	KIA	144	70	74	0	0.486	8.5
20	2017	1	KIA	144	87	56	1	0.608	0.0
21	2017	2	두산	144	84	57	3	0.596	2.0



Youngpyo Ryu. All rights reserved.
Unauthorized reproduction or distribution is prohibited.